# Resource selection functions

## Animal tracking 25/26

Giuliano Colosimo

University of Rome Tor Vergata

- These slides present a very brief introduction to Resource Selection Functions, a class of functions to introduce you to Species Distribution Modeling
- This is a rather complicated topic and requires some high level mathematical and coding skills
- Here we will just scratch the surface and try to run an analytical example using code borrowed from Björn Reineking and from the ctmm official website to try and understand how to link tracking data and environmental conditions

# Resource selection functions

- Resource selection functions (RSFs) are statistical models used in spatial ecology to assess which habitat characteristics are important to a specific animal population or species by estimating the probability of an animal using a resource relative to its availability in the environment
- How the environment (resources such as food, risks such as predation, conditions such as temperature) shapes movement paths and long-term utilization distributions (as a function of animal characteristics such as size, sex, diet)
- Predicts range distribution (fraction of time the animal spends at a given site in the long term)

- RSFs assume that successive positions (i.e. the locations along the track) are independent
- Good news: autocorrelation can be accounted for by weighing individual positions (Alston et al. 2023 MEE)
- Bad news: effective sample sizes are in general much lower than the number of tracking points (for buffalo Cilla: 3500 points at 1 hour interval translate to effective sample size of < 20)

# Setting up your computer

- To reproduce the code presented in these slides we first need to install all packages and data necessary to do so
- The packages and data needed can be installed using the following commands
- BE ADVISED: the installation of the following packages and dependencies is, per se, rather tricky and complicated, but it could be good practice to solve the issue that will arise during the installation process

```
install.packages("remotes")
remotes::install_github("AniMoveCourse/animove_R_package")
```

- Once all packages have been successfully installed we can proceed with loading the libraries of interest for this practical example

```
library(animove)
library(ctmm)
library(sf)
library(mvtnorm)
library(terra)
```

- The practical will use buffalo tracking data and link them to environmental data coded as raster files

# Load environmental data

```
data(buffalo_env)

# Convert from raster to terra object.
# terra is, currently, the recommended
# package to work with raster data.

buffalo_env <- rast(buffalo_env)

# plot the environmental data

plot(buffalo_env)
```

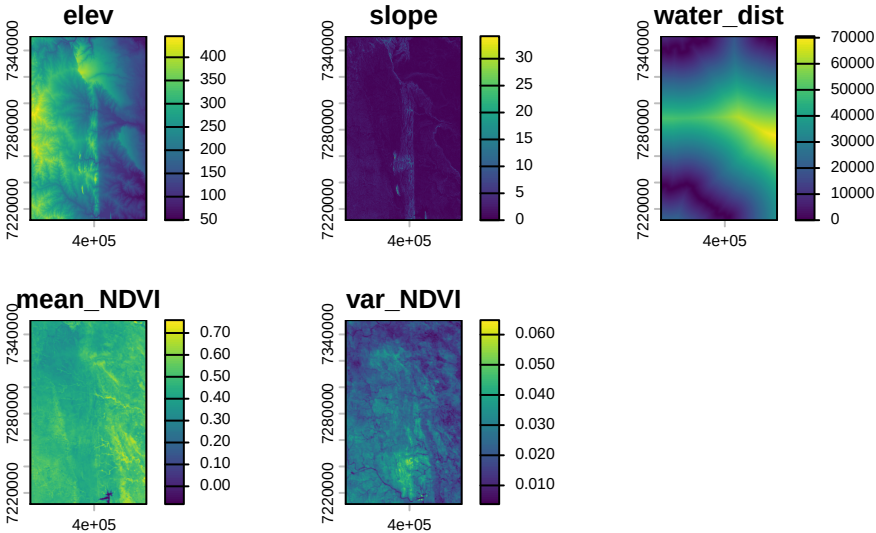


Figure 1: Environmental data. Topography, waterways and NDVI

# Load buffalo data

```
data(buffalo)

# project buffalo tracks to projection of raster layers

projection(buffalo) <- crs(buffalo_env)
```

- We can now proceed to plot the buffalo tracks against elevation

```
plot(buffalo_env[[1]])
# Convert to move2 object for plotting
buffalo_mv <- move2::mt_as_move2(buffalo)
plot(buffalo_mv, add = TRUE)
```

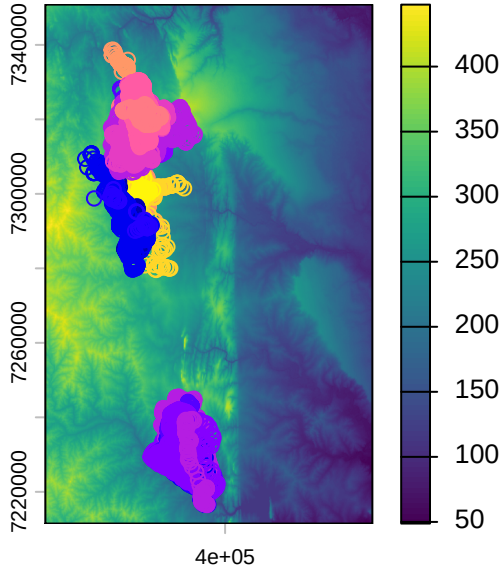


Figure 2: Animals tracks plotted on an elevation raster

- We can now subset the buffalo dataset and select only one individual. We will use the track for the individual called Cilla

```
cilla_mv <- subset(buffalo_mv, track == "Cilla")

plot(buffalo_env[[1]], ext = extent(cilla_mv) * 2)

plot(st_geometry(cilla_mv), add = TRUE, type = "p",
     col = "red", pch = 19, cex = 0.5)
```

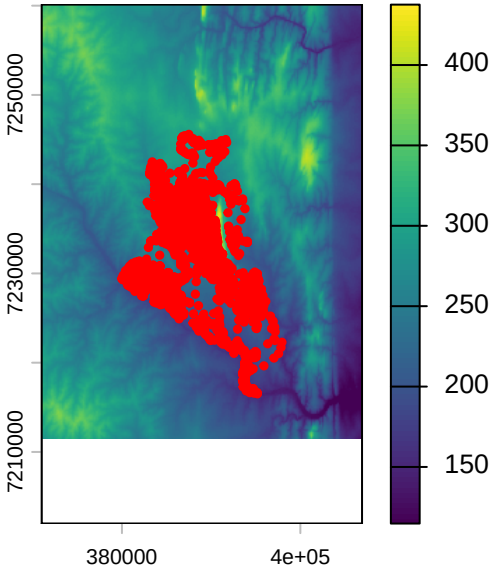


Figure 3: Cilla's tracking data

# Minimal example of resource selection

- Using the tracking data from Cilla we will now calculate the RSF and see how this animal uses the space and the resources in it
- This is a minimal example to introduce you to the complexity of this analysis
- The core library to calculate the RSF is ctmm (Calabrese, Fleming, & Gurarie, 2016), an R package for analyzing animal tracking data as a continuous-time stochastic processes
- We first produce a variogram, i.e. a representation of the spatial dependence between pairs of data points. This a very important first step as it will tell us something about the spatial autocorrelation of the data point we are about to analyse

```r
cilla <- buffalo$Cilla

SVF <- variogram(cilla)

# 50% and 95% CIs
level <- c(0.5,0.95)

# 0-12 hour window
xlim <- c(0,12 %#% "hour")

plot(SVF,fraction=0.65,level=level)
```

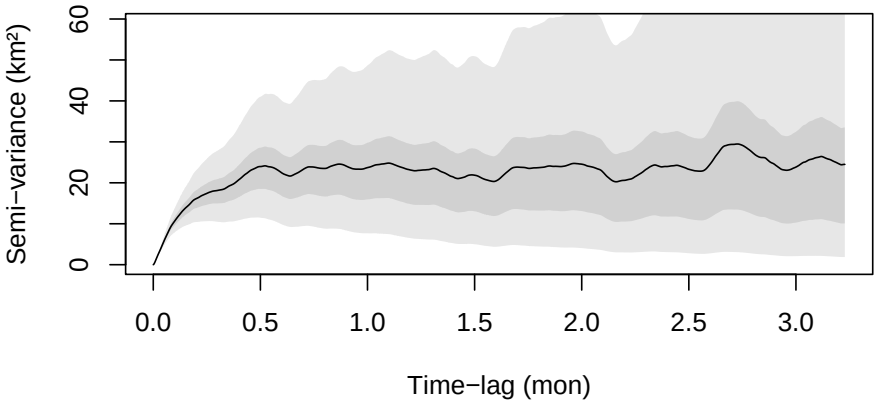


Figure 4: The variogram represents the average square distance traveled (vertical axis) within some time lag (horizontal axis)

- We can see that the variogram flattens (asymptotes) at approximately 20 days. This is, roughly, how coarse you need to make the timeseries so that methods assuming independence (no autocorrelation) can be valid. This includes, conventional kernel density estimation (KDE), minimum convex polygon (MCP), conventional species distribution modeling (SDM), and a host of other analyses
- We can now “fit” models to the variograms. This is not a rigorous statistical fitting, but a good way of choosing candidate movement models and guessing at their parameters. The variogram-fit parameter guestimates will then be fed into maximum likelihood estimation, which requires good initial guesses for non-linear optimization

```r
cilla_guess <- ctmm.guess(cilla,
                          CTMM = ctmm(isotropic = TRUE),
                          interactive = FALSE)

# Fit ctmm model and perform model selection,
# using all but one core
if (file.exists("cilla_fit_rsf.rds")) {
  cilla_fit <- readRDS("cilla_fit_rsf.rds")
} else {
  cilla_fit <- ctmm.select(cilla, cilla_guess,
                           cores = -1)
  saveRDS(cilla_fit, "cilla_fit_rsf.rds")
}
```

- Create named list of rasters. The function rsf.fit can deal with raster layers that differ in resolution or even projection

```
buffalo_env_list <-
  list("elev" = raster::raster(buffalo_env[["elev"]]),
  "slope" = raster::raster(buffalo_env[["slope"]]),
  "var_NDVI" = raster::raster(buffalo_env[["var_NDVI"]]))
```

- Definition of reference grid the akde (autocorrelated kernel density estimation) predictions should get aligned to

```
reference_grid <- crop(buffalo_env_list[["elev"]],
                       extent(cilla_mv) * 2)
```

- Fit and then plot akde

```
cilla_akde <- akde(cilla, cilla_fit,
                   grid = reference_grid)

plot(cilla_akde)
```

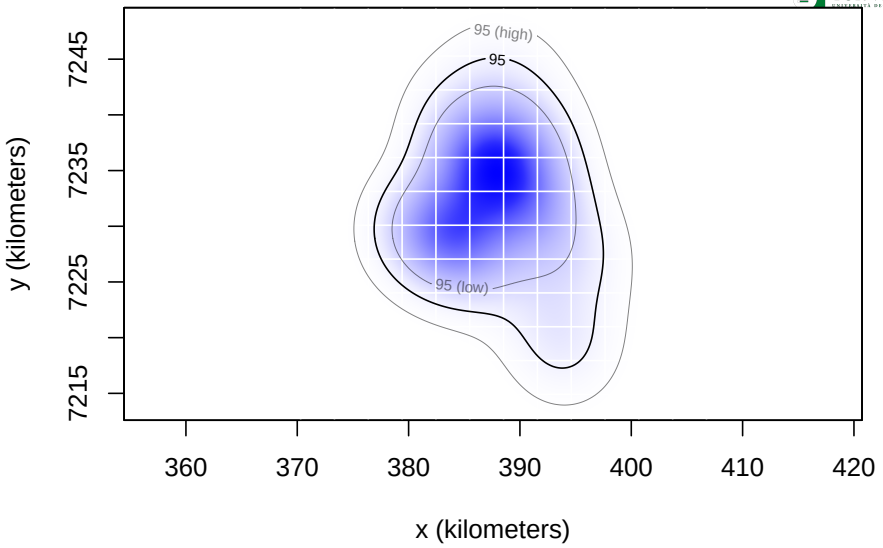


Figure 5: Autocorrelated kernel density estimation

- Create a raster representation of the akde, for the reference grid PMF is “probability mass function”. The values sum up to 1 for the whole grid, and represent for a given pixel the probability of finding the animal in that pixel

```
plot(ctmm::raster(cilla_akde, DF = "PMF"))
```

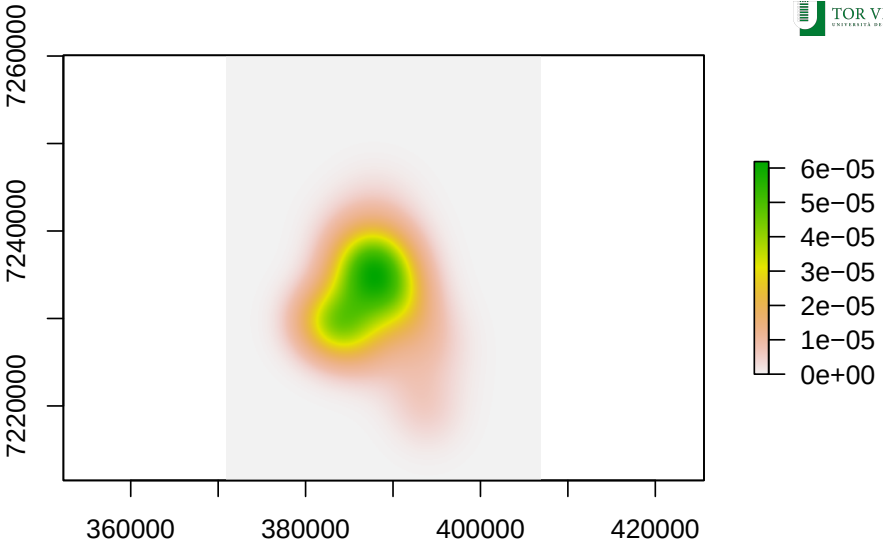


Figure 6: Probability mass function plot

```r
cilla_rsf_riemann <- rsf.fit(cilla, cilla_akde,
            R = buffalo_env_list, integrator = "Riemann")

suitability_riemann <-
  ctmm::suitability(cilla_rsf_riemann,
                    R = buffalo_env_list,
                   grid = reference_grid)

terra::plot(suitability_riemann)
```

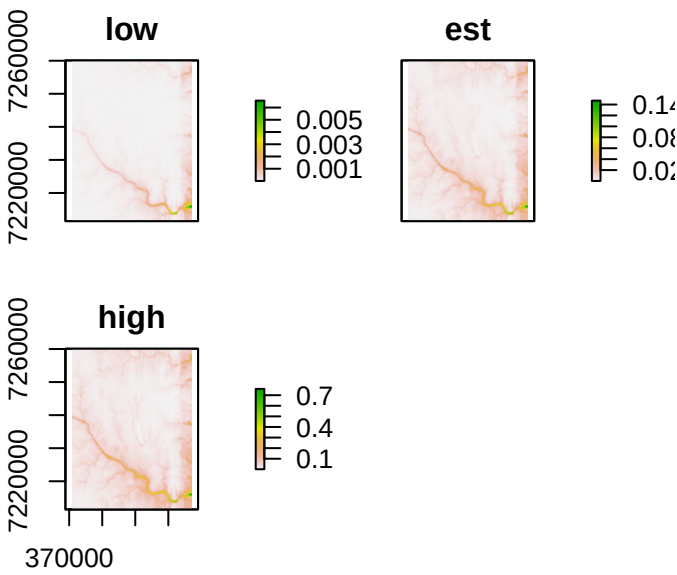


Figure 7: A suitability map - with 95% confidence interval

```
agde_cilla <- agde(CTMM = cilla_rsf_riemann,
                   R = buffalo_env_list,
                   grid = reference_grid)
agde_raster <- ctmm::raster(agde_cilla, DF = "PMF")
akde_rsf <- akde(cilla, CTMM = cilla_rsf_riemann,
                 R = buffalo_env_list,
                 grid = reference_grid)
pmf <- raster::stack(ctmm::raster(cilla_akde,
                                  DF = "PMF"),
              ctmm::raster(agde_cilla, DF = "PMF"),
              ctmm::raster(akde_rsf, DF = "PMF"))
names(pmf) <- c("akde", "rsf.fit", "akde_rsf.fit")
plot(pmf, zlim=c(0, max(raster::getValues(pmf))))
```

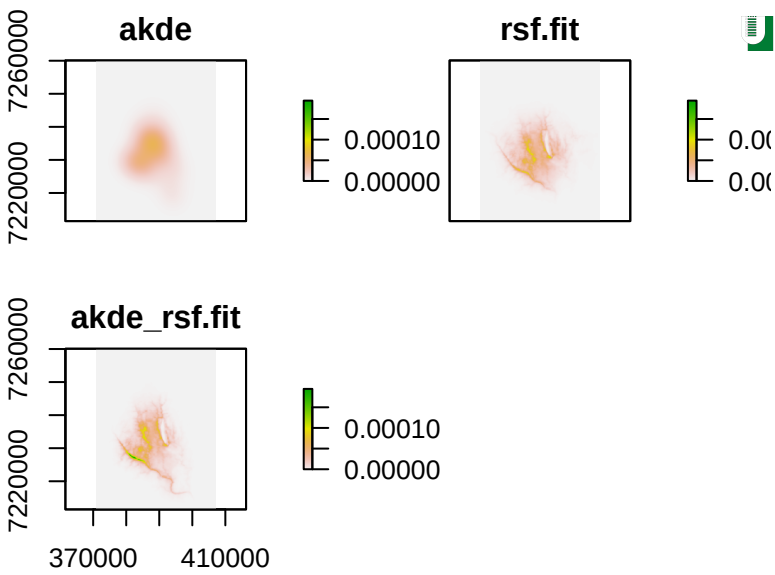


Figure 8: All three probability mass functions with a common color scale

# References I

Calabrese, J. M., Fleming, C. H., & Gurarie, E. (2016). Ctmm: An r package for analyzing animal relocation data as a continuous-time stochastic process. *Methods in Ecology and Evolution*, *7*(9), 1124–1132. Retrieved from https://besjournals.onlinelibrary.wiley.com/doi/abs/10.1111/2041-210X.12559